

Designing a DevSecOps Toolchain to Support ETL Workflows in Regulated Industries

Daniel Dickerson

November 5, 2025

Contents

1	Introduction	3
1.1	Context and Motivation	3
1.2	Problem Statement	3
1.3	Research Objectives	4
1.4	Contributions	4
1.5	Thesis Structure	4
2	Literature Review	6
2.1	Background	6
2.2	Defining Key Concepts	7
2.3	RQ1: ETL Pipelines in Regulated Industries	9
2.4	RQ2: DevSecOps and Governance in the SDLC	9
2.5	RQ3: Comparing the Data Lifecycle and the SDLC	10
2.6	RQ4: Tooling for Control Assurance and Governance	11
2.7	Synthesis and Related Works	13
3	Methodology	14
3.1	Design Science Context	14
3.2	Regulatory Context and Requirement Derivation	14
3.3	Requirements	15
3.4	Evaluation Framework	16
4	Baseline Pipeline	18
4.1	Overview	18
4.2	Architecture	18
4.3	Functionality	19
4.4	Observed Gaps	20
4.5	Summary	20
5	Proposed Architecture	22
5.1	Design Principles	22
5.2	High-Level Overview	22
5.3	Core Components	22
5.4	Integration with DevSecOps Practices	22
5.5	Expected Outcomes	22
6	Prototype Implementation	23
7	Evaluation	24
8	Discussion	25
9	Conclusion	26

Chapter 1

Introduction

1.1 Context and Motivation

In recent years, the value of data has surpassed that of the software systems built to process it [1, 2]. Organisations increasingly treat data as a product, complete with ownership structures, stewardship responsibilities, and lifecycle management frameworks [1]. This shift has coincided with growing regulatory scrutiny across jurisdictions [1]. Frameworks such as APRA’s Prudential Standards [3–6], alongside global instruments such as the EU’s GDPR and the United States’ HIPAA, demand not only the secure handling of information assets but also transparent processes that demonstrate accountability and control [2].

Extract–Transform–Load (ETL) workflows lie at the heart of this transformation, serving as the operational backbone of analytical systems [7, 8]. As Mahmud and Ikbāl [9] note, modern ETL pipelines have evolved into socio-technical infrastructures that sustain the scalability, reliability, and legitimacy of data-driven decision-making. Yet despite the rise of DataOps, many pipelines remain opaque, lacking end-to-end lineage, reproducibility, and verifiable controls [2]. In regulated environments, such opacity undermines compliance and erodes institutional trust.

Software engineering has already faced similar challenges. DevSecOps — the integration of development, security, and operations — has matured into a discipline embedding verification, automation, and governance across the software delivery lifecycle [10–13]. If data is now the product, DevSecOps offers an architectural and cultural foundation for treating data pipelines with the same rigour once reserved for software systems [14]. This research explores that proposition: how DevSecOps principles can be adapted to strengthen auditability and data governance within ETL workflows.

1.2 Problem Statement

While the DataOps movement has improved automation and observability, its primary focus remains operational efficiency rather than regulatory assurance [9, 15]. Existing orchestration frameworks, though powerful, rarely produce immutable audit trails, reproducible execution states, or cryptographically verifiable lineage. Auditability thus emerges as an accidental property rather than a designed one [16, 17]. As a result, organisations face a persistent gap between the technical execution of pipelines and the evidentiary requirements of regulators [3, 4].

This thesis addresses that gap. It investigates how a *DevSecOps-enabled toolchain* can enhance auditability and data governance across the ETL lifecycle. DevSecOps has already institutionalised

security and compliance for software artifacts, but an equivalent framework for data artifacts has yet to emerge. This absence exposes regulated organisations to compliance risk, weakens governance, and impairs their ability to demonstrate trustworthy data management.

1.3 Research Objectives

This research adopts a Design Science methodology [18] to design, implement, and evaluate a DevSecOps-based toolchain that enhances auditability and governance in ETL workflows. The study follows an iterative cycle of artifact design, prototype construction, and evaluation against explicit metrics for lineage completeness, reproducibility, and tamper detection.

Primary Research Question

How can a DevSecOps toolchain be designed to support ETL workflows in regulated industries, enhancing auditability and data governance?

In the process of answering the above question, this paper explores four sub-questions:

Sub-Questions

1. How do ETL workflows manifest in regulated industries, and what auditability and data governance challenges do they face?
2. How do DevSecOps practices enhance auditability and data governance within the software development lifecycle?
3. In what ways do data lifecycle management practices extend or reshape the software development lifecycle when DevSecOps principles are applied to ETL workflows in regulated environments?
4. What categories of tooling exist in DevSecOps and ETL pipelines, and how do their features support auditability and data governance?

1.4 Contributions

This thesis makes three primary contributions. First, it proposes an **architectural model** for a DevSecOps-enabled ETL toolchain, aligning the data lifecycle with secure software delivery practices to improve auditability and governance. Second, it presents a **prototype implementation** that integrates infrastructure as code, automated security scanning, and immutable logging into an operational ETL environment built with Airflow, Spark, and AWS components. Third, it defines an **evaluation framework** for quantifying auditability and governance through traceability, reproducibility, and tamper-resistance metrics. Together, these contributions demonstrate how DevSecOps principles can be embedded in data-centric systems to enhance compliance and trust.

1.5 Thesis Structure

The remainder of this thesis is organised as follows:

- **Chapter 2** reviews literature on ETL pipelines, data governance, and DevSecOps practices.
- **Chapter 3** explains the Design Science methodology, design constraints, and evaluation criteria.
- **Chapter 4** describes the baseline ETL pipeline and identifies auditability and governance gaps.
- **Chapter 5** presents the proposed DevSecOps-integrated architecture.
- **Chapter 6** outlines the prototype implementation and toolchain configuration.
- **Chapter 7** evaluates the system against defined auditability and governance metrics from Chapter 3.
- **Chapter 8** discusses implications, limitations, and avenues for future research.
- **Chapter 9** concludes with reflections on how DevSecOps can enable trustworthy data governance in regulated environments.

Chapter 2

Literature Review

This chapter reviews the body of knowledge relevant to the design of ETL pipelines in regulated industries and the application of DevSecOps principles. It begins by outlining the context of ETL and governance, defines the key concepts central to this research, and then examines the literature associated with each research question.

2.1 Background

Data pipelines underpin modern analytical systems, forming the connective infrastructure between data sources and analytic platforms [14]. They transform raw data into governed, traceable data products used in applications such as fraud detection, market analysis, compliance reporting, and metadata processing.

Although the extract–transform–load (ETL) process appears simple in theory, it is in practice complex, resource-intensive, and often the most time-consuming element of data warehousing [19]. ETL pipelines act as the “eyes of the system,” feeding reliable data downstream; when lineage is lost or visibility fails, the reliability of the entire process is compromised [8]. Vayyala [8] emphasises that maintaining end-to-end lineage across transformations is vital for sustaining trust in analytics-driven decisions.

In regulated industries, compliance depends as much on how data is processed as on the outcomes produced. Frameworks such as GDPR, SOX, and APRA’s CPS standards require secure, ethical, and well-documented data handling [1, 3–6]. These frameworks place accountability, ownership, and an end-to-end evidence chain at the centre of compliance culture.

Auditability, in this context, is the ability to demonstrate compliance through reproducibility, lineage, and tamper-evident records [2]. Trustworthy analytics rely not only on integrity of data but also on assurance evidence of correct process execution. Governance provides the organisational scaffolding for this assurance—linking policy to technical enforcement through defined roles, procedures, and controls [1, 2]. Failures in either dimension expose organisations to legal, financial, and reputational risk [1].

DevOps emerged to accelerate software delivery through automation, continuous integration and deployment (CI/CD), and rapid feedback cycles [13]. DevSecOps extends this model by embedding security and compliance across the software development lifecycle. Practices such as automated testing, compliance-as-code, artifact signing, and tamper-evident logging produce assurance evidence that can be inspected by regulators [10, 11, 13]. This convergence bridges engineering and governance concerns, establishing control assurance as a built-in property of the delivery process.

As data itself becomes a primary strategic asset, surpassing the value of the systems that process it [1, 2], the mechanisms of DevSecOps must be extended to the data pipelines that sustain organisational decision-making.

2.2 Defining Key Concepts

The following subsections define the key concepts underpinning this research: DevSecOps, ETL, data governance, and auditability. Each definition draws from existing literature and sets the theoretical foundation for later analysis.

DevSecOps

DevSecOps is the proactive and continuous enforcement of security and compliance requirements throughout the software development lifecycle (SDLC) [10]. Unlike traditional models that treat security as an external stage or afterthought, DevSecOps embeds it within every phase of the lifecycle [12]. Compliance becomes the expected outcome by default, and deviations from defined standards must be explicitly justified [11].

DevSecOps is notoriously difficult to define succinctly. It is often described as the sum of its implementation patterns—CI/CD, infrastructure-as-code (IaC), policy-as-code (PaC), software scanning, automated tests, and GitOps. A more practical lens is to examine the gap created when DevSecOps is removed: the ungoverned space between code that compiles and a production service. In that gap, identity and approval are ad hoc, policy enforcement is manual, change control is episodic, evidence is incomplete, and configuration drifts without reconciliation. DevSecOps closes this gap by making control assurance continuous, placing verification at the point of change, and generating the lineage and evidence metadata needed for independent assessment [10, 20].

A defining characteristic of DevSecOps is its reliance on automation to enforce non-functional requirements such as control assurance and governance. Through automated testing, provisioning, monitoring, and policy enforcement, systems produce assurance evidence—signed artifacts, tamper-evident records, and control indicators—that regulators and auditors can independently assess [10, 21]. This automated assurance narrows the gap between delivery teams and governance stakeholders [12].

By combining cultural expectations of shared responsibility with technical mechanisms for verification, DevSecOps establishes a delivery model in which security and compliance are continuous, measurable, and intrinsic—rather than externally imposed.

ETL

Extract–Transform–Load (ETL) is a structured process that ingests, transforms, and persists data into target repositories such as data warehouses, lakes, or file stores [19]. It traditionally comprises three stages: extraction, where data is sourced from heterogeneous systems; transformation, where it is cleansed, standardised, integrated, and reshaped into coherent semantic structures; and loading, where transformed data is stored for downstream analysis [19]. Kimball and Caserta describe ETL as both the most resource-intensive and most critical link between operational data and analytical use.

Modern practice extends beyond this classical batch model. Approaches such as extract–load–transform (ELT) and hybrid streaming–batch architectures leverage distributed and cloud-native infrastructure [9]. ETL now encompasses not only data integration and quality but also governance and compliance. In regulated industries, pipelines must guarantee lineage, reproducibility, and control assurance in addition to performance and scalability [9].

ETL is thus best understood as a flexible socio-technical process. It concludes when data is made persistent and reusable but intersects with broader lifecycle concerns such as orchestration, monitoring, and policy enforcement. In this sense, ETL pipelines underpin both analytical scalability and institutional trust [9].

Data Governance

Data governance provides the strategic framework that defines how data is collected, managed, and protected across an organisation. It establishes the policies, roles, standards, and processes that ensure information assets remain accurate, consistent, secure, and compliant with internal and external regulations [1, 2]. Nai, Rifai, and Sadiq [1] describe governance as a structured framework that ensures the availability, integrity, usability, and security of data through stewardship and accountability roles.

Vayyala [2] expands this understanding by positioning governance as the foundation of reliable data ecosystems—linking policy to technical enforcement through automation, lineage tracking, and tamper-evident records. In practice, governance institutionalises accountability so data products can withstand regulatory scrutiny.

Auditability

Auditability is the designed capability of a system to allow its processes, controls, and outcomes to be observed and verified by stakeholders. Weigand and Elsas [17] define auditability through five interdependent requirements:

- **Normativity:** a clear regulatory or contractual framework that governs operations.
- **Accountability:** verifiable statements linking actions to responsible agents and governing norms.
- **Referentiality Infrastructure:** records that connect operations to their original points of recording.
- **Reliability:** faithful and tamper-resistant representation of events.
- **Verifiability:** the ability of external parties to independently test or validate claims.

Although these principles originate in organisational and financial control, they can be transferred to information systems. Extending this theoretical foundation, Wang et al. [16] demonstrate how auditability can be operationalised through third-party verification, cryptographic proofs of integrity, and support for dynamic data operations. Their work illustrates that auditability is not merely an organisational goal but a property that can be engineered into data management infrastructures.

Applied to ETL pipelines and DevSecOps practices, these five dimensions map directly to regulated-industry concerns:

- **Normativity:** embedding compliance and data-protection policies within workflow execution.
- **Accountability:** linking each transformation or deployment to authenticated agents or automated processes.
- **Referentiality Infrastructure:** ensuring full lineage tracking across extraction, transformation, and loading.
- **Reliability:** maintaining tamper-evident records, signed artifacts, and cryptographic hashes.

- **Verifiability:** enabling reproducibility tests and independent control testing via CI/CD-integrated checks.

Together, these adapted requirements frame auditability as a designed and engineered property of ETL systems. Every data operation and system change must attach to an end-to-end evidence chain that is reproducible and independently verifiable against regulatory expectations.

2.3 RQ1: ETL Pipelines in Regulated Industries

ETL pipelines in regulated industries operate within a constant tension between agility and accountability. They are expected to deliver analytical value with speed and scale, yet must simultaneously produce verifiable evidence that every data movement complies with institutional and regulatory constraints. The modern pipeline has therefore evolved into a socio-technical infrastructure—one that mediates trust between engineers, auditors, and regulators [8, 9]. Its success cannot be judged solely by throughput or latency; rather, by its ability to maintain lineage, transparency, and compliance under continuous change.

Financial, healthcare, and critical-infrastructure systems exemplify this dual demand. Frameworks such as APRA’s CPS 230, CPS 234, and CPS 510 [3–5] institutionalise separation of duties and controlled promotion of change, translating ethical responsibility into technical procedure. These standards do not exist apart from engineering practice—they shape it—embedding legal accountability into operational design. Governance scholarship reaches similar conclusions, emphasising that tamper-evident logging, role-based accountability, and retention aligned with compliance lifecycles form the practical machinery of institutional trust [1, 2, 7]. Cloud-native orchestrators such as Airflow, AWS Glue, and Azure Data Factory provide elasticity and automation, but they operate under the shadow of these mandates. Their convenience is constrained by the need to preserve traceability and access control consistent with CPG 235 [6]. In this context, orchestration itself becomes a form of governance: a choreography of tasks and dependencies that leaves a verifiable trail of execution [22].

Over time, this operational discipline has turned the pipeline from a means of data transport into a living archive of control assurance. Continuous validation, versioning, and policy enforcement integrate assurance logic directly into the runtime environment [2]. Metadata management forms the connective tissue between policy and process, ensuring that lineage and evidence metadata, schema evolution, and transformation logic can be reconstructed and defended [7]. Yet even within mature systems, full control assurance remains elusive. Toolchain heterogeneity fragments lineage across layers of orchestration, computation, and storage. Business logic embedded in SQL or user-defined functions escapes formal tracing, while schema drift and inconsistent metadata propagation corrode reliability [9]. Few frameworks create tamper-evident, cryptographically verifiable artifacts by default [7, 17]. What is missing is not information, but integrity—the ability to prove that recorded operations are both complete and untampered. The fragility of this end-to-end evidence chain provides the practical motivation for integrating DevSecOps assurance into ETL environments, turning validation from an external audit to an intrinsic system property.

2.4 RQ2: DevSecOps and Governance in the SDLC

The software development lifecycle (SDLC) treats change as a controlled and observable process. Standards such as ISO/IEC/IEEE 12207 and 15288 define the artifacts, procedures, and documentation that make each phase accountable [23, 24]. Governance in this model is not an external overlay—it is embedded in the way systems are planned, built, and released.

DevSecOps strengthens this principle by embedding assurance, security, and compliance across every

phase of delivery [10, 20]. The delivery pipeline becomes a generator of assurance evidence: each build, test, and deployment produces signed artifacts, tamper-evident records, and compliance manifests that bind design intent to execution [10, 12, 17, 25]. These artifacts collectively form a continuous end-to-end evidence chain that makes system state traceable and verifiable at any point in time.

Automation enables this continuity. Infrastructure-as-code (IaC), continuous-integration/continuous-delivery (CI/CD), and policy-as-code (PaC) apply verification directly at the point of change. Static and dynamic testing, dependency scanning, and approval gates transform compliance into an embedded control validation signal rather than a periodic audit [12, 24, 26]. The NIST Secure Software Development Framework codifies this expectation: traceable metadata, lineage and evidence metadata, and policy enforcement are standard lifecycle outputs [20]. Mature build systems act as lineage and evidence-metadata engines, linking requirements, commits, and runtime behaviour within the same evidentiary record.

As software supply chains expand, continuous verification of component origin and integrity becomes essential [20, 26]. Toolchains respond through automated scanning, provenance analysis, and AI-assisted policy enforcement [27, 28]. Requirement traceability then links regulatory expectations directly to runtime artifacts [23, 25], ensuring that what is built, deployed, and reviewed aligns to the same authoritative record [13].

Several implementation patterns operationalise DevSecOps principles of automation and assurance. Infrastructure-as-Code (IaC) enforces reproducible, governed environments; Policy-as-Code (PaC) embeds compliance validation within delivery pipelines; and GitOps extends these mechanisms into runtime operations by reconciling deployed state with version-controlled configuration. Together, these patterns close the loop between design-time verification and operational control, forming the practical foundation of continuous control monitoring within the SDLC.

2.5 RQ3: Comparing the Data Lifecycle and the SDLC

The data lifecycle (DLC) aims for control and reliability but lacks the SDLC’s formal standardisation. Wing describes the DLC as a dynamic, context-dependent model rather than a fixed sequence of phases [29]. Most frameworks share the same goal: to keep data reliable, accessible, and verifiable throughout its lifespan [30, 31]. The SDLC governs change and transformation; the DLC governs persistence and stewardship. One manages evolution, the other preserves continuity.

Despite their structural differences, the two lifecycles can be aligned. The Data Management Body of Knowledge (DAMA-DMBOK) provides one of the most widely adopted codifications of the DLC, framing governance through roles, metadata catalogues, lineage records, and quality assessments that together establish an end-to-end evidence chain [32]. These governance artifacts closely mirror the information items defined in ISO/IEC/IEEE 15289 [23], demonstrating that the logic of software assurance maps naturally onto data management practice. DevSecOps supplies the mechanisms for this connection: continuous validation and automated security controls apply equally to code and data [10, 20]. Lineage manifests, schema definitions, and transformation scripts can therefore be versioned and audited alongside software components.

Encryption, secrets management, and role-based access add cryptographic assurance to this framework [14, 33]. Agile governance pushes data validation earlier in the lifecycle [7, 8, 34], mirroring the shift in DevSecOps toward early verification. These expectations reflect APRA’s prudential standards [3–6], where assurance is evidenced through the operational trace rather than through static reports.

When the SDLC and DLC intersect, they create a shared evidentiary framework. ISO/IEC/IEEE 15289’s artifacts—plans, specifications, procedures, and reports—extend to include lineage documentation, schema evolution, and validation outputs. IaC and policy-as-code make these records executable [12, 13, 35]. Reproducibility, tamper detection, and governance then operate through the same

automated pipelines that deliver software.

In regulated ETL workflows, control assurance becomes a first-class design property. Each transformation of code or data leaves a signed, tamper-evident record that shows what occurred, who performed it, and why it was authorised.

2.6 RQ4: Tooling for Control Assurance and Governance

Assurance depends on implementation. The ability to trace, verify, and reproduce each operational event relies on tools that enforce control, capture lineage, and preserve evidence of change. In both DevSecOps and ETL environments, tooling translates regulatory requirements such as those in CPG 235 and CPS 234/510 into machine-verifiable artifacts [3, 5, 6].

This section examines the tool categories that make such evidence possible. In DevSecOps, frameworks for infrastructure-as-code, policy validation, and continuous integration generate signed artifacts and lineage and evidence metadata that prove system integrity. In ETL pipelines, orchestration, metadata management, and validation platforms create traceable datasets and transformation histories that support data governance. Each tool category contributes to the same end-to-end evidence chain linking software and data processes into a single, verifiable system of record.

DevSecOps tooling: from code to operations

Infrastructure- and Policy-as-Code

IaC and PaC convert infrastructure and policy into versioned artifacts so that compliance checks run before change. Terraform, CloudFormation, OPA, Checkov, and Conftest generate explicit evidence of desired state and policy evaluation [10, 33]. Key contrasts appear in Table 2.1.

Table 2.1: IaC/PaC tools and the evidence they produce

Tool	Strengths	Limitations
Terraform	Versioned infra; plan/apply logs; OPA integration	Limited native provenance
CloudFormation	CloudTrail linkage; tight IAM controls	Vendor lock-in
OPA / Conftest	Declarative policy; decision logs as evidence	Rule complexity
Checkov	IaC scanning; SBOM export	Limited contextual correlation

CI/CD orchestration

GitHub Actions, GitLab CI, Jenkins, and Azure Pipelines attach identity, review, and signatures to every run; pipeline logs form part of the evidence chain [20, 25]. Table 2.2 summarises key trade-offs.

Table 2.2: CI/CD systems as provenance engines

Tool	Strengths	Limitations
GitHub Actions	Signed runs and artifacts; Git provenance	Coarse access audit
GitLab CI	Built-in approvals; compliance reports	Higher ops cost
Jenkins	Extensible audit plugins	Plugin sprawl
Azure Pipelines	RBAC tie-in; policy checks	Cloud specific

SAST, DAST, and SCA

Security scanners create structured evidence of vulnerabilities and dependency lineage. They integrate with CI/CD pipelines and feed policy gates [36, 37]. Table 2.3 summarises trade-offs.

Table 2.3: Security scanning tools and audit artifacts

Tool	Strengths	Limitations
SonarQube	Persistent quality/issue history	Needs tuning; limited container context
OWASP ZAP	Reproducible web tests; open formats	Lower throughput at scale
Snyk	Dependency risk; SBOM generation	Proprietary backend; partial offline
Trivy	Unified SCA/SAST; SBOM export	Narrow policy correlation scope

Testing and validation

Test frameworks provide repeatable proof of expected behaviour; their outputs become verifiable artifacts [20, 37]. Table 2.4 outlines key examples.

Table 2.4: Testing frameworks as reproducible evidence

Framework	Strengths	Limitations
Postman / Newman	Versioned API suites; runnable logs	Manual upkeep of collections
PyTest	Parametrised fixtures; stable outputs	Python-only scope
JUnit	Mature CI integration; clear reports	No native audit metadata

Containers and runtime

Images, admission controls, and runtime events contribute directly to system integrity evidence [10, 13]. Table 2.5 lists tools and their audit characteristics.

Table 2.5: Container and runtime controls as integrity evidence

Tool	Strengths	Limitations
Docker	Image digests and signatures	Limited fine-grained runtime policy
Kubernetes	Declarative policy; drift detection	Multi-cluster audit complexity
Falco	Real-time event evidence	Rule maintenance overhead
Prisma Cloud / Sysdig	Centralised compliance view	Proprietary; cost

Monitoring and observability

Control indicators and logs form the final evidence layer, supporting incident reconstruction and trend analysis [10, 33]. Table 2.6 highlights typical stacks.

Table 2.6: Observability stacks and audit support

Tool	Strengths	Limitations
Prometheus	Transparent, timestamped metrics	High cost for long retention
Grafana	Clear dashboards; evidence views	Dependent on source data quality
ELK / Splunk	Searchable forensic logs	Operational cost; index complexity

Convergence

GitOps as an Operational Assurance Pattern

GitOps is a practical implementation of DevSecOps principles that operationalises configuration and change assurance. By declaring desired system state in version control and automating reconciliation, GitOps enforces configuration provenance, segregation of duties, and continuous control validation across environments. In this unified model, the same controls that attest software integrity also attest data integrity. Approval gates in CI/CD govern the promotion of ETL workflows; policy checks constrain data transfers; and container attestations record the runtime context of both applications and transformations. Observability systems link these artifacts into a continuous audit timeline spanning code, infrastructure, and data [10, 20, 38]. GitOps enforces configuration provenance; complementary lineage controls extend this assurance to data artifacts and transformations.

These implementation patterns—particularly GitOps—form the operational basis of the proposed architecture in Chapter 5, where declarative configuration and automated reconciliation underpin continuous assurance across both software and data lifecycles. Regulatory expectations under CPG 235 and CPS 234/510 [3, 5] are satisfied not through separate audits but through a single, declarative lifecycle.

Three assurance patterns make this convergence practical. **Single-path promotion** ensures that every artifact—application, configuration, or dataset—follows the same reviewed path to production. **Policy-guarded reconciliation** binds control objectives to each applied change. **Attested runtime** couples container and orchestration evidence to specific commits and policies, closing the loop between declared intent and observed outcome [10, 20].

2.7 Synthesis and Related Works

Research at the intersection of DevSecOps and data governance converges on the same foundation—automation, traceability, and verifiable control—but few works treat these as parts of a single evidence system spanning the full ETL lifecycle.

The IGI Global collection *Data Governance, DevSecOps, and Advancements in Modern Software* [1] provides the strongest conceptual foundation. Its chapters explain how metadata management, lineage, and policy-as-code can bridge security automation and governance accountability. Vayyala [7] presents metadata as a governance mechanism; Kose [34] links agile compliance to DevSecOps; and Soussane [14] applies these principles to real-time analytics. Collectively, they demonstrate how DevSecOps can embed governance within delivery pipelines, though they stop short of implementing tamper-evident and reproducible evidence across the data lifecycle.

Subsequent research in software assurance [10, 20] and lifecycle management [29, 39] reinforces these principles but remains domain-bound—software or data, not both. This thesis advances that junction by proposing a unified DevSecOps-enabled ETL toolchain that integrates assurance and governance into one continuous, verifiable end-to-end evidence chain for regulated industries.

Chapter 3

Methodology

This chapter outlines the methodology used to design, construct, and evaluate a DevSecOps toolchain that governs ETL workflows in regulated environments. The research applies a Design Science Research (DSR) approach, chosen for its focus on building artifacts that address real-world problems while generating transferable knowledge [18]. Rather than developing a new ETL pipeline, this study designs the system that governs such pipelines—the architecture, control automation, and controls that embed assurance and compliance into operational data workflows.

3.1 Design Science Context

Design Science generates knowledge through the purposeful creation and evaluation of artifacts [40]. It follows an iterative build–evaluate process in which understanding arises from constructing, implementing, and refining solutions. In this study, the artifact is a *DevSecOps governance and assurance framework*—a set of integrated components that enforce policy, record lineage and evidence metadata, and assure integrity across an ETL pipeline. The pipeline itself functions as a *reference pipeline* that represents a typical control-baseline environment within a regulated domain.

This methodological choice is appropriate because Design Science balances *rigour*—alignment with regulatory and theoretical principles—with *relevance*—demonstrated utility in practice. Each design iteration: problem identification, artifact construction, demonstration, and evaluation, corresponds to a development stage where the framework’s assurance features are incrementally refined and tested. This cyclical process mirrors the continuous control monitoring model advocated in DevSecOps, where systems evolve through assurance evidence and feedback.

3.2 Regulatory Context and Requirement Derivation

The toolchain design is guided by the Australian Prudential Regulation Authority’s (APRA) Prudential Standards: CPS 230 (*Operational Risk Management*), CPS 234 (*Information Security*), and CPS 510 (*Governance*) [3–5]. These standards define what regulated entities must achieve in governance, security, and resilience, but they do not prescribe how those objectives should be implemented technically. Their descriptive, outcomes-based language demands interpretation into enforceable engineering controls.

Although the methodology is domain-agnostic, this research focuses on the *financial sector*. Finance offers a mature and well-documented regulatory landscape that provides both conceptual depth

and practical relevance for governance evaluation. APRA’s prudential standards form a coherent, enforceable set of requirements that align with the objectives of assurance, lineage, and accountability identified in the literature. This focus reflects the local regulatory environment and the availability of structured compliance expectations that make the sector suitable for Design Science evaluation.

Accordingly, the scope is narrowed to a practical subset of CPS requirements most relevant to data-pipeline governance. The selected principles—continuity, accountability, and control—are translated into implementable requirements for control automation and evidence generation. This refinement is informed by the literature review in Chapter 2, which frames auditability (as defined there) in terms of lineage, evidence, and verification across socio-technical systems.

3.3 Requirements

The following requirements define what the DevSecOps framework must achieve to operationalise prudential expectations. They establish the conditions under which the framework can govern a pipeline by embedding control, lineage, and continuous assurance.

1. **R1: Governance Accountability (CPS 510).** The framework must preserve authorship and approval lineage for all operational and configuration changes, ensuring full traceability of decision authority and execution.
2. **R2: Information Security Assurance (CPS 234).** The framework must maintain artifact integrity and confidentiality through tamper-evident storage, cryptographic verification, and continuous security scanning (SCA, SAST, and DAST).
3. **R3: Operational Risk Control (CPS 230).** The framework must detect, record, and remediate process or configuration failures, maintaining reproducibility and recovery within defined assurance thresholds.
4. **R4: Continuous Control Monitoring (CPS 234, 510).** The framework must automate policy validation and control testing in the CI/CD process, producing machine-verifiable assurance evidence through policy-as-code enforcement.
5. **R5: End-to-End Evidence Chain (CPS 230).** The framework must generate and retain a tamper-evident, queryable record that captures the lineage of code, configuration, and data artifacts throughout the system lifecycle.

These requirements shift focus from the business logic of ETL processes to the *meta-layer* of assurance—how evidence is produced, tested, and preserved. They align with the five auditability dimensions defined in Section 2.2: normativity, accountability, referentiality, reliability, and verifiability.

Table 3.1: Mapping between toolchain requirements and auditability dimensions.

Requirement	Normativity	Accountability	Referentiality	Reliability	Verifiability
R1 Governance Accountability	●	●			
R2 Information Security Assurance				●	●
R3 Operational Risk Control			●	●	
R4 Continuous Control Monitoring	●	●			
R5 End-to-End Evidence Chain			●		●

Governance and control monitoring (R1, R4) express the normative and accountable aspects of control; security and risk management (R2, R3) reinforce reliability and traceability; and the end-to-end evidence chain (R5) provides verifiable, referential links across artifacts and executions. Together, these create a governance and assurance framework where assurance is continuous and evidence is intrinsic.

3.4 Evaluation Framework

Evaluation in Design Science verifies that an artifact fulfils its intended purpose. Here, the goal is to confirm that the DevSecOps framework meets each requirement (R1–R5). Each requirement becomes an evaluation unit with defined control indicators, data sources, and assurance thresholds.

R1 Governance Accountability

Objective: Confirm that all changes are traceable to authenticated identities and documented approvals.

Method: Examine change events, approval records, and version metadata for completeness and signature validity.

Control Indicators:

- 100% of configuration and code changes linked to a verified identity.
- 100% of recorded approvals associated with tamper-evident artifacts.
- Continuous lineage maintained across system versions.

Auditability Dimensions: Normativity, Accountability.

R2 Information Security Assurance

Objective: Demonstrate that artifacts and configurations are verified for integrity and security.

Method: Conduct automated and manual reviews confirming that artifacts are signed, validated, and free from known vulnerabilities.

Control Indicators:

- 100% of artifacts validated through checksum or signature verification.
- Zero undetected tampering events during testing.
- Complete retention of verification records.

Auditability Dimensions: Reliability, Verifiability.

R3 Operational Risk Control

Objective: Validate reproducibility and recovery from prior workflow states.

Method: Execute controlled replays of historical states and compare outputs to reference baselines.

Control Indicators:

- $\geq 95\%$ lineage completeness between input, process, and output.
- Successful re-execution of three or more historical states.
- Recovery times within defined assurance thresholds.

Auditability Dimensions: Referentiality, Reliability.

R4 Continuous Control Monitoring

Objective: Verify that compliance enforcement and validation operate automatically for every change.

Method: Observe multiple workflow executions to confirm policy checks and validations occur consistently.

Control Indicators:

- $\geq 95\%$ of executions meet all control criteria.
- No unapproved or unverified changes reach production.
- All control evaluations generate verifiable records.

Auditability Dimensions: Normativity, Accountability.

R5 End-to-End Evidence Chain

Objective: Confirm that the framework maintains a tamper-evident, queryable record of artifacts and lineage.

Method: Perform end-to-end tracing of artifacts across executions to ensure each output can be reconstructed from recorded evidence.

Control Indicators:

- 100% of evidence and lineage records validate without error.
- All transformations traceable to original inputs.
- Zero discrepancies in signature or hash verification tests.

Auditability Dimensions: Referentiality, Verifiability.

Evaluation Synthesis

Each requirement is tested through direct observation and measurement, with results mapped back to the auditability dimensions in Table 3.1. Collectively, these evaluations determine whether the governance and assurance framework establishes a verifiable end-to-end evidence chain across its lifecycle. The analysis confirms not only functional correctness but also the degree to which the artifact embodies the principles of continuous assurance defined in Chapter 2.

Chapter 4

Baseline Pipeline

This chapter outlines the baseline ETL pipeline that serves as a control model for later evaluation. It demonstrates a minimal, functional data workflow built with industry-standard components but without any embedded governance or assurance features. The baseline provides the empirical ground for measuring how DevSecOps practices enhance auditability in the proposed system.

4.1 Overview

The pipeline extracts public data from the SEC EDGAR API, stages it to Amazon S3 as raw CSV, transforms it with Spark on EMR, and writes curated Parquet outputs to S3. Airflow, running on a Dockerised EC2 instance, orchestrates the workflow. Infrastructure is deployed through Terraform and continuously integrated with GitHub Actions, which builds the Airflow image, provisions resources, and triggers deployments on each `push` to `main`.

Scope. The configuration is intentionally simple. It omits authentication hardening, formal lineage capture, or immutability controls. Defaults are accepted where possible to keep the environment lightweight, reproducible, and comparable to the secured prototype that follows.

4.2 Architecture

The system operates across three planes: orchestration, compute, and storage. Figure 4.1 shows the arrangement of these components within AWS. Airflow on EC2 acts as the control plane, submitting Spark jobs to EMR and coordinating their execution. The data plane spans two S3 buckets—one for raw ingestion and one for curated outputs—linked by Spark transformations.

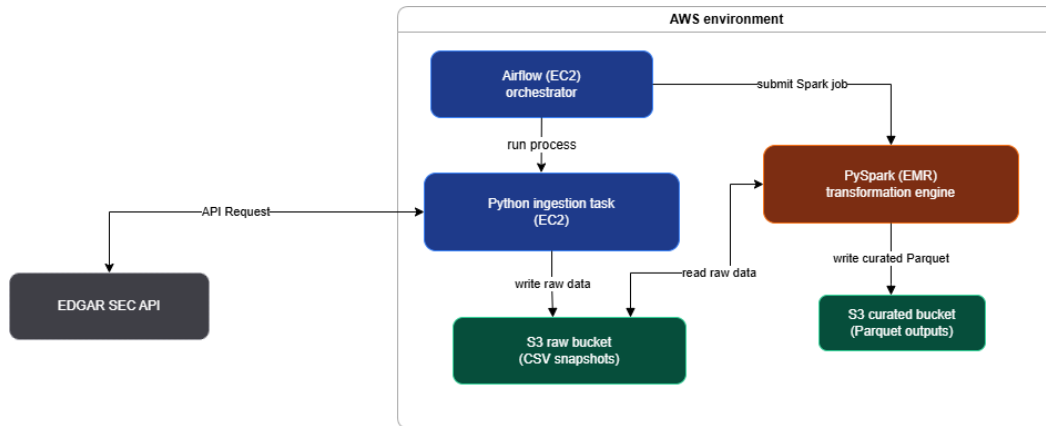


Figure 4.1: Baseline runtime architecture showing orchestration, compute, and storage components.

Continuous integration and deployment are automated through GitHub Actions. Figure 4.2 illustrates the CI/CD workflow: syntax checks precede infrastructure provisioning via Terraform, after which the Airflow image—which embeds the DAGs—is built, pushed to GHCR, and redeployed to the EC2 host.

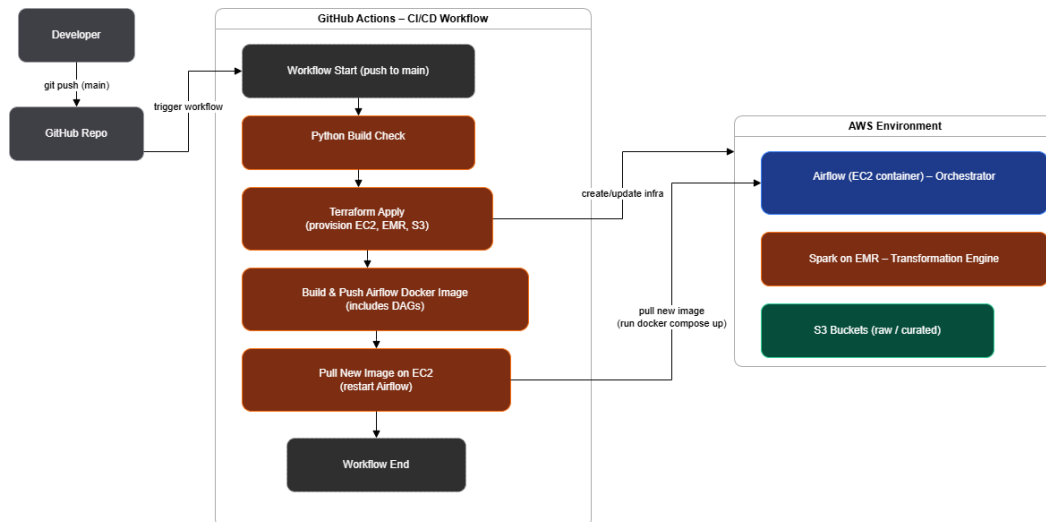


Figure 4.2: CI/CD pipeline treated as a DAG within GitHub Actions.

4.3 Functionality

Figure 4.3 presents the Airflow DAG that governs the ETL workflow. A single linear sequence performs ingestion, submission of Spark jobs, and post-run manifest recording.

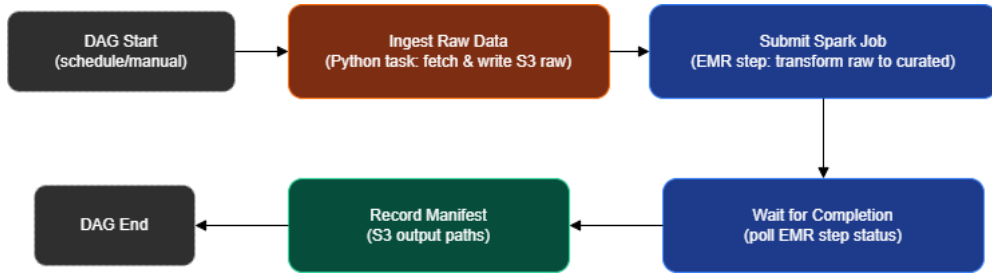


Figure 4.3: Baseline Airflow DAG showing orchestration sequence.

Data moves through three primary stages (Figure 4.4). The ingestion task fetches source files and stores them in the raw S3 bucket. Spark reads these raw objects, enforces schema consistency, cleans and deduplicates records, and writes Parquet datasets to the curated bucket, partitioned by ingestion date. Each run emits a simple manifest (`_SUCCESS`) listing output paths for downstream validation.

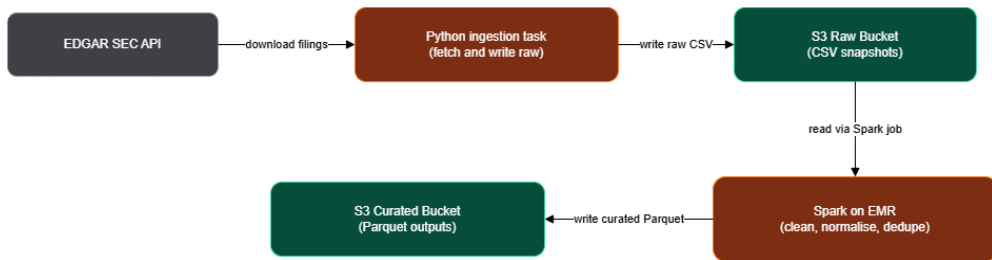


Figure 4.4: Data path from ingestion to curated output in the baseline pipeline.

4.4 Observed Gaps

While the baseline performs correctly, it lacks the controls required for trustworthy audit evidence.

Lineage. Provenance is implicit in log files rather than captured as structured metadata. No mapping links a specific input object, transformation script, and output dataset.

Reproducibility. Environment state is defined by mutable container images. There is no guarantee that a re-run reproduces prior results byte-for-byte, nor any version pinning for dependencies or configuration.

Governance and Security. IAM policies are broad, and actions are not gated by review or policy validation. Objects in S3 remain mutable and unsigned, and there is no mechanism to detect tampering or verify artifact integrity.

4.5 Summary

The baseline ETL pipeline demonstrates a functioning but ungoverned data workflow. It provides consistent data movement from extraction to curated storage, yet its assurance relies on convention

rather than design. The next chapter extends this model with explicit DevSecOps controls—embedding lineage tracking, immutable evidence, and governed infrastructure to meet auditability expectations.

Chapter 5

Proposed Architecture

5.1 Design Principles

Outline the guiding principles of the proposed DevSecOps-integrated ETL system: automation, immutability, traceability, reproducibility.

5.2 High-Level Overview

Provide an architectural diagram comparing baseline and proposed design.

5.3 Core Components

Explain each layer:

- **Infrastructure as Code:** Terraform for provisioning.
- **Orchestration:** Airflow for controlled execution.
- **Security:** CI/CD checks, artifact signing, SAST/SCA.
- **Governance:** Immutable audit logs, metadata tracking.

5.4 Integration with DevSecOps Practices

Map the proposed pipeline to the DevSecOps lifecycle (plan → code → build → test → release → monitor).

5.5 Expected Outcomes

Summarize the architectural advantages for auditability and compliance.

Chapter 6

Prototype Implementation

To be completed in Thesis B

Chapter 7

Evaluation

To be completed in Thesis B

Chapter 8

Discussion

To be completed in Thesis B

Chapter 9

Conclusion

To be completed in Thesis B

Bibliography

- [1] S. Nai, A. Rifai, and A. Sadiq, “Data Governance, Key Insights, Strategic Challenges, and Future Imperatives;,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 1–16. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch001>
- [2] R. Vayyala, “Ensuring Data Quality and Integrity in Modern Enterprises;,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 17–46. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch002>
- [3] Australian Prudential Regulation Authority, “Prudential Standard CPS 234: Information Security,” Canberra, ACT, Jul. 2019. [Online]. Available: https://www.apra.gov.au/sites/default/files/cps_234_july_2019_for_public_release.pdf
- [4] —, “Prudential Standard CPS 230: Operational Risk Management,” Canberra, ACT, Jul. 2025. [Online]. Available: <https://www.apra.gov.au/sites/default/files/2025-07/Prudential%20Standard%20CPS%20230%20Operational%20Risk%20Management%20-%20clean.pdf>
- [5] —, “Prudential Standard CPS 510: Governance,” Canberra, ACT, Jul. 2023. [Online]. Available: https://www.apra.gov.au/sites/default/files/prudential_standard_cps_510_governance.pdf
- [6] —, “Prudential Practice Guide CPG 235: Managing Data Risk,” Australian Prudential Regulation Authority (APRA), Canberra, ACT, Prudential Practice Guide CPG 235, Sep. 2013. [Online]. Available: https://www.apra.gov.au/sites/default/files/Prudential-Practice-Guide-CPG-235-Managing-Data-Risk_1.pdf
- [7] R. Vayyala, “Metadata Management and Its Role in Data Governance;,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 47–72. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch003>
- [8] —, “Data Lineage: Tracing Data Across Systems,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 73–98. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch004>
- [9] D. Mahmud and M. Z. Ikbali, “THE ROLE OF ETL (EXTRACT-TRANSFORM-LOAD) PIPELINES IN SCALABLE BUSINESS INTELLIGENCE: A COMPARATIVE STUDY OF DATA INTEGRATION TOOLS,” *ASRC Procedia: Global Perspectives in Science and Scholarship*, vol. 2, no. 1, pp. 89–121, Apr. 2022. [Online]. Available: <https://global.asrcconference.com/index.php/asrc/article/view/29>
- [10] M. Anisetti, N. Bena, F. Berto, and G. Jeon, “A DevSecOps-based Assurance Process for Big Data Analytics,” in *2022 IEEE International Conference on Web Services (ICWS)*, Jul. 2022, pp. 1–10. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9885738>
- [11] C. Castellanos, C. A. Varela, and D. Correal, “ACCORDANT: A domain specific-model and DevOps approach for big data analytics architectures,” *Journal of Systems and Software*, vol.

172, p. 110869, Feb. 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121220302594>

- [12] M. Anisetti, C. A. Ardagna, E. Damiani, and F. Gaudenzi, “A Semi-Automatic and Trustworthy Scheme for Continuous Cloud Service Certification,” *IEEE Transactions on Services Computing*, vol. 13, no. 1, pp. 30–43, Jan. 2020. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7831357/authors>
- [13] P. Pandey and A. Patel, “Integrating Security in Cloud-Native Development: A DevSecOps Approach to Resilient Software Systems,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 169–196. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch009>
- [14] K. Soussane, B. E. Elbaghazaoui, and M. Amnai, “Integrating Security in DataOps: A Framework for Securing Data Pipelines in Real-time Analytics,” in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 197–214. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch010>
- [15] A. R. Munappy, D. I. Mattos, J. Bosch, H. H. Olsson, and A. Dakkak, “From Ad-Hoc Data Analytics to DataOps,” in *Proceedings of the International Conference on Software and System Processes*, ser. ICSSP ’20. New York, NY, USA: Association for Computing Machinery, 2020, pp. 165–174, event-place: Seoul, Republic of Korea. [Online]. Available: <https://doi.org/10.1145/3379177.3388909>
- [16] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, “Enabling Public Auditability and Data Dynamics for Storage Security in Cloud Computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 22, no. 5, pp. 847–859, May 2011. [Online]. Available: <https://ieeexplore.ieee.org/document/5611497>
- [17] H. Weigand and P. Elsas, “Auditability as a Design Problem,” in *2019 IEEE 21st Conference on Business Informatics (CBI)*, vol. 01, Jul. 2019, pp. 276–285, iSSN: 2378-1971. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8808092>
- [18] A. Hevner, A. R. S. March, S. T. Park, J. Park, Ram, and Sudha, “Design Science in Information Systems Research,” *Management Information Systems Quarterly*, vol. 28, pp. 75–, Mar. 2004.
- [19] R. Kimball and J. Caserta, *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. Wiley, 2004, iISBN: 9780764567575. [Online]. Available: <https://www.oreilly.com/library/view/the-data-warehouse/9780764567575/>
- [20] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication (SP) 800-218, Feb. 2022. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/218/final>
- [21] Y. Zhou, Y. Yu, and B. Ding, “Towards MLOps: A Case Study of ML Pipeline Platform,” in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, Oct. 2020, pp. 494–500. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9361315>
- [22] A. Pogiatis and G. Samakovitis, “An Event-Driven Serverless ETL Pipeline on AWS,” *Applied Sciences*, vol. 11, no. 1, p. 191, Jan. 2021, publisher: Multidisciplinary Digital Publishing Institute. [Online]. Available: <https://www.mdpi.com/2076-3417/11/1/191>
- [23] “ISO/IEC/IEEE International Standard – Systems and software engineering - Content of life-cycle information items (documentation),” Jul. 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8767110>
- [24] R. Hernández, B. Moros, and J. Nicolás, “Requirements management in DevOps environments: a multivocal mapping study,” *Requirements Engineering*, vol. 28, no. 3, pp. 317–346, Sep. 2023. [Online]. Available: <https://doi.org/10.1007/s00766-023-00396-w>

- [25] T. Chen and H. Suo, "Design and Practice of Security Architecture via DevSecOps Technology," in *2022 IEEE 13th International Conference on Software Engineering and Service Science (ICSESS)*, Oct. 2022, pp. 310–313, iSSN: 2327-0594. [Online]. Available: <https://ieeexplore.ieee.org/document/9930212>
- [26] J. Li and H. Li, "Evolution of Application Security based on OWASP Top 10 and CWE/SANS Top 25 with Predictions for the 2025 OWASP Top 10," in *2025 International Conference on Inventive Computation Technologies (ICICT)*, Apr. 2025, pp. 1178–1183, iSSN: 2767-7788. [Online]. Available: <https://ieeexplore.ieee.org/document/11004742>
- [27] A. Mittal and V. Venkatesan, "Practical Integration of Large Language Models into Enterprise CI/CD Pipelines for Security Policy Validation: An Industry-Focused Evaluation," in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, Jul. 2025, pp. 197–203, iSSN: 2642-6587. [Online]. Available: <https://ieeexplore.ieee.org/document/11126149>
- [28] B. Achuthan and M. A. Alimohideen, "Shifting Gears: Integrating Security Audits into Automotive DevSecOps," in *2024 International Conference on Vehicular Technology and Transportation Systems (ICVTTS)*, vol. 1, Sep. 2024, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/10763937>
- [29] J. M. Wing, "The Data Life Cycle," *Harvard Data Science Review*, vol. 1, no. 1, Jul. 2019, publisher: The MIT Press. [Online]. Available: <https://hdr.mitpress.mit.edu/pub/577rq08d/release/4>
- [30] A. Ball, *Review of Data Management Lifecycle Models*. Bath, UK: University of Bath, Feb. 2012.
- [31] W. C. Lenhardt, S. Ahalt, B. Blanton, L. Christopherson, and R. Idaszak, "Data Management Lifecycle and Software Lifecycle Management in the Context of Conducting Science | Journal of Open Research Software," Feb. 2014. [Online]. Available: <https://openresearchsoftware.metajnl.com/articles/10.5334/jors.ax>
- [32] P. Cupoli, S. Earley, and D. Henderson, "DAMA-DMBOK2 Framework."
- [33] A. Patel and P. Pandey, "Safeguarding Sensitive Data in the DevSecOps Pipelines: Strategies, Tools, and Best Practices for Modern Software Development," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 215–240. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch011>
- [34] B. O. Kose, "Agility Meets Compliance: The Convergence of Data Governance and DevSecOps," in *Data Governance, DevSecOps, and Advancements in Modern Software*, B. E. Elbaghazaoui, M. Amnai, and N. Gherabi, Eds. IGI Global, Apr. 2025, pp. 131–154. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/979-8-3373-0365-9.ch007>
- [35] H. Myrbakken and R. Colomo-Palacios, "DevSecOps: A Multivocal Literature Review," in *Software Process Improvement and Capability Determination*, A. Mas, A. Mesquida, R. V. O'Connor, T. Rout, and A. Dorling, Eds. Cham: Springer International Publishing, 2017, pp. 17–29.
- [36] D. B. Cruz, J. R. Almeida, and J. L. Oliveira, "Open Source Solutions for Vulnerability Assessment: A Comparative Analysis," *IEEE Access*, vol. 11, pp. 100 234–100 255, 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10251527>
- [37] T. Rangnau, R. v. Buijtenen, F. Franssen, and F. Turkmen, "Continuous Security Testing: A Case Study on Integrating Dynamic Security Testing Tools in CI/CD Pipelines," in *2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC)*, Oct. 2020, pp. 145–154, iSSN: 2325-6362. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9233212>
- [38] S. Gupta, M. Bhatia, M. Memoria, and P. Manani, "Prevalence of GitOps, DevOps in Fast CI/CD Cycles," in *2022 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COM-IT-CON)*, vol. 1, May 2022, pp. 589–596. [Online]. Available: <https://ieeexplore.ieee.org/document/9850786>

- [39] J. Zhang, J. Symons, P. Agapow, J. T. Teo, C. A. Paxton, J. Abdi, H. Mattie, C. Davie, A. Z. Torres, A. Folarin, H. Sood, L. A. Celi, J. Halamka, S. Eapen, and S. Budhdeo, “Best practices in the real-world data life cycle,” *PLOS Digital Health*, vol. 1, no. 1, p. e0000003, Jan. 2022, publisher: Public Library of Science. [Online]. Available: <https://journals.plos.org/digitalhealth/article?id=10.1371/journal.pdig.0000003>
- [40] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007, publisher: Taylor & Francis, Ltd. [Online]. Available: <https://www.jstor.org/stable/40398896>